

INSTITUTO TÉCNICO SUPERIOR ESPECIALIZADO
SEDE TOCUMEN

INNOVACIÓN DIGITAL
Técnico Superior en Desarrollo de Software



Tecnología de la Información y Comunicación

PROFESOR:

AARON ANTHONY SMITH PANEZO

TALLER 1-GRUPAL

“Evolución de las TIC y su Impacto en el Desarrollo de Software”

TEMA #1: Evolución de los Lenguajes de Programación

ESTUDIANTE & CÉDULA:

Gaitán Yorlenis *8-984-633*

GRUPO:

D-DS-1-1

FECHA DE ENTREGA:

10 de junio de 2026

I CUATRIMESTRE

INTRODUCCIÓN

El Inicio

La máquina del telar de Jacquard, creada en 1801, utilizaba los orificios en tarjetas perforadas para representar los movimientos de un brazo de la máquina de tejer, con el objetivo de generar patrones decorativos automáticamente, este fue el primer paso para la generación y paso de datos. Entre 1842 y 1843, Ada Lovelace tradujo las memorias del matemático italiano Luigi Menabrea acerca de la nueva máquina propuesta por Charles Babbage, la Máquina Analítica (computador moderno de uso general). Con estos escritos, ella añadió unas notas en las cuales especificaba en detalle un método para calcular los números de Bernoulli (sucesión de números racionales profundamente conectados con la teoría de números, el análisis matemático y el cálculo de sumas de potencias, buscaba la simplificación de grandes potencias) con esta máquina.

Definiciones importantes para la comprensión del tema:

Según su Nivel de Abstracción:

- Lenguajes de Bajo Nivel: Están muy cerca del hardware.
 - *Lenguaje de máquina*: Compuesto por código binario (0 y 1).
 - *Ensamblador (Assembler)*: Usa abreviaturas (mnemónicos) para comunicarse directamente con la CPU. (mov: Copia el dato de un lugar a otro, add: El procesador realiza la suma de dos valores y almacena el resultado siempre en el operando de destino, sub: lo contrario a add, lo resta)
- Lenguajes de Alto Nivel: Utilizan una sintaxis más parecida al lenguaje humano, lo que facilita su comprensión y portabilidad.
- Lenguajes de Nivel Medio: Ofrecen un equilibrio entre el control del hardware y la legibilidad humana.

Según su Paradigma:

- Orientados a Objetos: Utilizan "clases" y "objetos" para modelar el mundo real.
- Imperativos: Se basan en dar al equipo instrucciones paso a paso sobre *cómo* cambiar el estado del programa. Ejemplos: C, PHP.
- Declarativos: Se enfocan en describir *qué* resultado se quiere obtener, en lugar de detallar el proceso paso a paso.

Según su Ejecución:

- Lenguajes Compilados: Un traductor convierte todo el código a lenguaje de máquina antes de ejecutar el programa. Suelen ser más rápidos. Ejemplos: C++, Rust.
- Lenguajes Interpretados: Un programa (intérprete) lee y ejecuta el código línea por línea en tiempo real.

INDICE

El Inicio	2
Primera Generación	5
1940	5
ENIAC – 1945.....	5
Plankalkül – 1946	6
Segunda Generación	7
1950	7
ALGOL 60 – 1958.....	7
Tercera Generación	9
1960	9
<i>HARDWARE GRAN PASO AL SOFTWARE DEL HOY</i>	9
BASIC – 1964	9
Cuarta Generación	10
1970	10
PASCAL – 1970	10
C – 1972	12
Quinta Generación	13
1980	13
<i>LENGUAJE DE ALTO NIVEL</i>	13
C++ – 1983.....	13
1990	14
Python – 1991.....	14
Java – 1995.....	16
Años 2000 y Actualidad	17
2000	17
Scala – 2004.....	17
Clojure – 2007.....	18
Dart – 2011	20
Análisis critico	22
CONCLUSIÓN	24
INFOGRAFÍA	25

Primera Generación

1940

ENIAC – 1945

(Computador e integrador numérico electrónico) Unas de las primeras computadoras de propósito general, diseñada inicialmente para calcular tablas de tiro de artillería para el ejército de los Estados Unidos, calculaban el ángulo de inclinación, relación entre el alcance de un arma, la carga del propulsor y condiciones meteorológicas que podían afectar en el lanzamiento del tiro. El cálculo balístico que antes requería 12 horas con una calculadora manual podía realizarse en tan solo 30 segundos. Esto significa que la ENIAC era 1440 veces más rápida.

ENIAC ocupaba una sala de 9 por 15 metros y pesaba 30 toneladas. Un total de 40 paneles estaban dispuestos en forma de U, con una longitud frontal de 24 metros, y los 18 000 tubos de vacío necesarios eran más de 20 veces superiores al total empleado por todos los sistemas a bordo de un bombardero B-29 en tiempos de guerra. Los circuitos de ENIAC incluían 500 000 uniones soldadas, con 70 000 resistencias y 10 000 condensadores. ENIAC también contaba con su propia línea eléctrica dedicada y consumía 150 kilovatios de electricidad.

FUNCIONAMIENTO:

Las características operativas de ENIAC incluían elementos aritméticos, de memoria y de control. En cuanto a las operaciones aritméticas, contaba con veinte acumuladores para suma y resta, un multiplicador y un divisor combinado con función de raíz cuadrada. Los elementos de memoria se componían de una memoria interna formada por dos partes: los veinte acumuladores de diez dígitos y los 6000 interruptores y cables en las tablas de

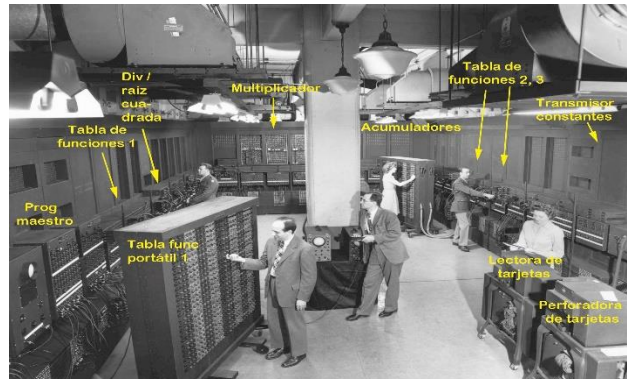


1 Pila de 62,500 tarjetas perforadas que en total equivalen a apenas 4.5 megabytes de datos informáticos

funciones, acumuladores y transmisores de constante. La memoria externa, de capacidad ilimitada, se alimentaba desde fuera de la máquina mediante tarjetas perforadas.

Aunque esta computadora en si no realizaba lenguajes de programación modernos si tuvo su propia resolución de problemas para lograr la eficiencia de las respuestas, ya que se tenían que estar cambiando cables constantemente crearon los primeros códigos primitivos:

- ENIAC Coding System: El Sistema de cableados (inicios de ENIAC)
- ENIAC Short Code: Permitieron a los programadores escribir expresiones matemáticas usando una notación simbólica, reemplazando la tediosa configuración física de cables y enchufes. (1949)

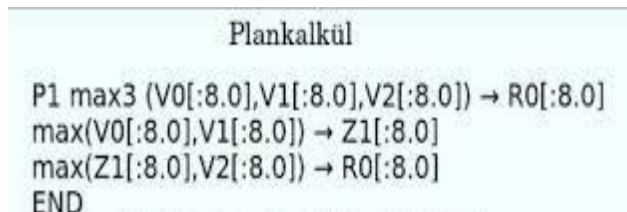


2 ENIAC

Con el último punto derivo el BINAC: Estaba formada por dos procesadores. Todas las instrucciones se ejecutaban por separado en ambos procesadores, luego se comparaban los resultados obtenidos, si eran iguales se continuaba con la ejecución de la siguiente instrucción, si en cambio eran distintos se paraba la ejecución del programa.

Plankalkül – 1946

Es un lenguaje de programación imperativo de alto nivel, con su propia creación e Z3, considerado el primer computador programable, empezó a trabajar en un lenguaje de programación que pudiera expresar algoritmos complejos y describir circuitos lógicos, para esto creo el sistema de notación combinatoria de condiciones. Sin embargo, por las condiciones sociopolíticas que involucraban su país tuvo que detener sus progresos en el área del hardware y se enfocó en el software, se enfocó en el Plankalkül.



3 Cálculo del máximo entre 3 variables de entrada

Tipos de datos:

1. Tipo primitivo único: un solo bit denominado Ja Nein-Werte, valores si-no. (S0)

Desde aquí se derivan la construcción de números enteros y reales

- Números enteros: $n \times S0$: 8.0 para entero 8bits
- Números reales: con punto flotante y un esquema similar al bit implícito que se usa actualmente en IEEE 754

2. Arrays: Matrices de dimensiones arbitrarias
3. Registros: Estructuras de datos complejos que permitían agrupar variables de diferentes tipos de datos bajo una misma unidad.

Variables:

Variable	Nombre alemán	Función
V	<i>Variablen</i>	Parámetros de entrada (paso de argumentos)
Z	<i>Zwischenwerte</i>	Valores intermedios (variables locales/temporales)
R	<i>Resultatwerte</i>	Resultados de salida

La primera asignación a una variable se consideraba su declaración. No existían declaraciones previas: el tipo se especificaba en el momento de uso.

- Asignación: =
- Condicional: →
- Interacción: W
- Sentencia Fin: hoy conocido como break, con superíndice para saltar fuera de varios niveles de anidamiento de bucles.
- Sin salto de líneas, quiere decir sin GOTO.

¿Dónde se encontraba el Plankakül?

- Primer programa completo para jugar ajedrez, aplicado finalmente en el año 1998 gracias a la creación de un compilador para este lenguaje.
- Creó representaciones de redes lógicas aplicadas a la resolución de problemas relacionales.

Segunda Generación

1950

ALGOL 60 – 1958

Aunque ALGOL es el lenguaje primario de esta gran familia, el enfoque será para ALGOL 60. ALGOL viene de la abreviatura ALGORITHMIC, quiere decir que es un lenguaje algorítmico especificado del año 60 (creado realmente en 1958), esta introdujo lo que

son los bloques de código (grupo de instrucciones o declaraciones), esto permite controlar de forma lógica el flujo de ejecución y delimitar el alcance de las variables, además de los pares que permitían la delimitación dando paso a lo que conocemos como la programación estructurada.

También con este lenguaje se introdujo los comandos de BEGIN y END, además de que proporcionó dos estrategias de evaluación para el paso de parámetros: el paso por valor común y el paso por nombre. La declaración del procedimiento especificaba, para cada parámetro formal, cuál se iba a usar: valor especificado para paso por valor, y omitido para paso por nombre. El paso por nombre recalcula cada parámetro cuando se usa en una expresión.

Variables o identificadores:

• ARRAY	• ELSE	• IF	• REAL	• TRUE
• BEGIN	• END	• INTEGER	• STEP	• UNTIL
• BOOLEAN	• FALSE	• LABEL	• STRING	• VALUE
• COMMENT	• FOR	• OWN	• SWITCH	• WHILE
• DO	• GOTO	• PROCEDURE	• THEN	

A medida que se fue actualizando y modificando ha ido aumentando la cantidad llegando a ser hasta 71 variables.

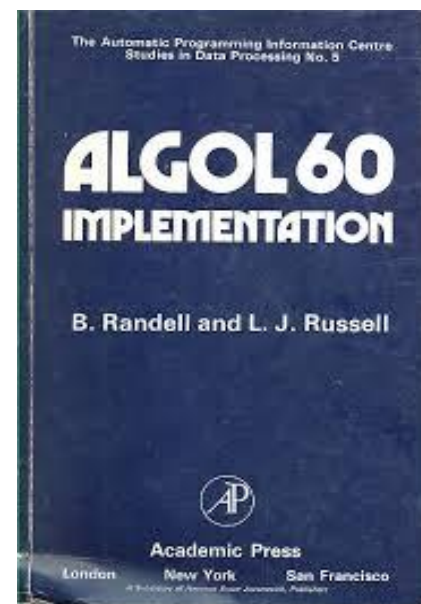
Lenguajes derivados de ALGOL 60:

- C
- PASCAL
- B

Utilización del lenguaje ALGOL60:

- Idioma universal de la ACM por más de 30 años
- Como base para los siguientes compiladores

Los compiladores se utilizan para transformar código fuente de alto nivel en código de destino de bajo nivel (como lenguaje ensamblador, código objeto o código de máquina) mientras se preserva la funcionalidad del programa.



4 Autor: B. Randell y L. J. Russell, Manual de ALGOL60

No incluyó funciones nativas o estándar de entrada y salida (I/O) para leer tarjetas perforadas o imprimir en pantallas de manera universal lo cual lo hacía incompatible. Como cada fabricante de hardware tuvo que inventar su propia solución improvisada de

comandos de lectura y escritura, un código escrito para una computadora no corría directamente en otra distinta.

Tercera Generación

1960

HARDWARE GRAN PASO AL SOFTWARE DEL HOY

Gracias a la evolución de los hardware evoluciono a la vez los softwares; antes diseñados eran para una sola tarea en específico debido a las limitantes físicas, viendo el problema se mejorando los procesadores, época en la cual pasamos de los tubos de vacío a los transistores dando como resultado a los circuitos integrados. Esto redujo significativamente el costo de las computadoras, su tamaño y consumo de energía.

Como resultado de esta invención comenzaron a crear software multitareas surgiendo el concepto de tiempo compartido, la *pseudo-simultaneidad*, da la ilusión de ejecutar múltiples tareas de forma simultánea, realmente el CPU está saltando de una tarea a otra rápidamente. Todo esto mediante el *scheduler* (programa encargado de administrar los recursos cuando deben de ser ejecutado en distintas tareas o procesos)

BASIC – 1964

Programación para el Alcance Todos

Por sus siglas en inglés:
***Beginners All-purpose
Symbolic Instruction Code***
(Código simbólico de instrucciones de propósito general para principiantes), creado por John G. Kemeny, Thomas E. Kurtz y Mary K. Keller en Estados Unidos.



5 Sesión de Comandos Commodore BASIC

Contextualizando el entorno, para esos años los las computadores eran únicamente usadas por aquellos que supieran codificar el software que estaba hecho a medida, estos eran únicamente los científicos y matemáticos, este lenguaje fue hecho para que los estudiantes fuesen capaces de utilizar los ordenadores. BASIC como muchos lenguajes de programación fue inicialmente creado para la educación y facilitador, gracias a esto este lenguaje y sus variantes fueron ampliamente divulgados por los siguientes 20 años entre las microcomputadoras. Esto dio pie a la creación de nuevos lenguajes basados en BASIC como Microsoft Visual Baic, GNU/Linux.

Basado en parte por FORTRAN II y Algol 60 modificado específicamente a los tiempos compartidos mediante múltiples terminales y facilitar la aritmética de matrices.

FUNCIONAMIENTO:

Solo necesitaba de los comandos LET, PRINT, INPUT, GOTO y IF. Al principio la numeración de las líneas era completamente manual, para poder así llamarlas o utilizarlas, para esto se iba de 10 en 10 para poder ingresar líneas entre medio de las instrucciones. Gracias a esto los programadores más expertos crearon el RENUMBER.

¿Dónde se vio utilizado el BASIC?

- 1975 desarrollo de Altair 8800 usando el lenguaje BASIC – Primeros Micrordenadores comerciales – Microsoft

Lenguajes derivados:

- Visual BASIC
- C
- Pascal

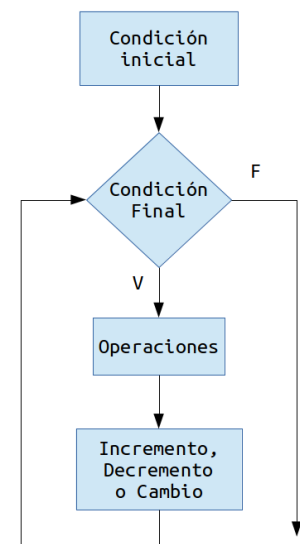
Cuarta Generación

1970

Continuando con la programación estructurada de los años 60, y de dejando de lado la transferencia incondicional GOTO, este sistema se ve compuesto por estructuras de control como:

- Sequence: Declaraciones ordenadas o subrutinas (procedimientos, funciones)
- Selection: 1 o varias instrucciones que se ejecutan dependiendo del programa.
- Iteration: Una instrucción o bloque sé que ejecuta hasta que el programa alcance cierto estado. (while, for, do, repeat).
- Recursion: declaración que se ejecuta hasta que se cumpla las condiciones de terminación.

Visto como un diagrama de flujo.

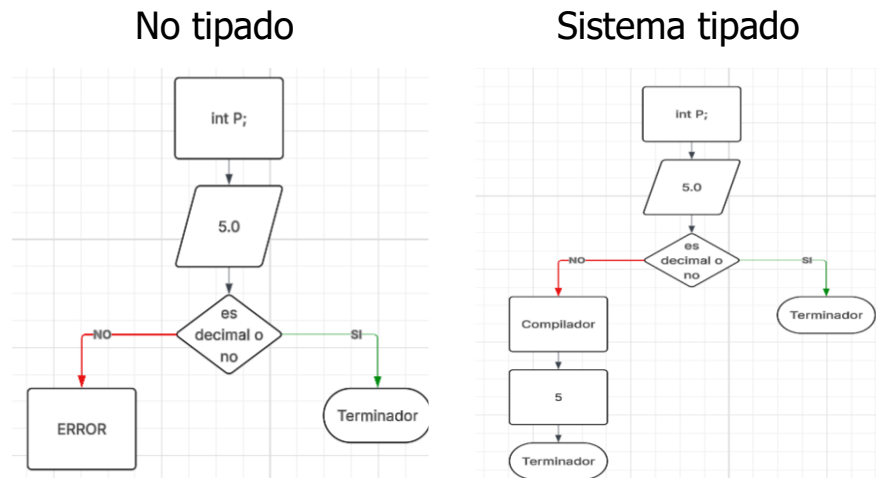


6 Diagrama de flujo ilustrativo

PASCAL – 1970

Creado por Niklaus Wirth para el aprendizaje de programación estructurada, caracterizado por su lenguaje de programación donde no se permite las violaciones de los tipos de datos, si se le da un valor a dicha variable no se puede usar como si fuese

otro tipo de variable, a menos que se haga la conversión.



7 Diagramas de flujos Sistemas Tipados

Ejemplo donde definimos que la P es una variable entera, en un lenguaje de programación no tipeado donde no puede conversión de variables, en cambio en un sistema tipado el programador debe de hacerlo explícitamente y debe de ser soportado por el compilador.

Fue una de las primeras versiones en ser un traductor universal, debido a que las computadoras eran de sistemas dependientes a los propietarios, tenían un solo código de máquina, se tenía que reescribir todo el código del software nuevamente si se quería usar en otra maquina diferente. Con esto llego el intermediario: La Máquina-P.

FUNCIONAMIENTO DE TRADUCTOR:

La máquina-P simulaba ser una computadora ficticia entonces:

- Universal: un único programa traducía el código PASCAL (código-P)
- Local: se crea un programa que lo interpreta en cada computadora real, este programa hacia creer que era la máquina-P a la computadora real.

¿Dónde fue utilizado el PASCAL?

- Apple Lisa
- Apple II y III
- Primeros Macintosh

RESULTADOS:

- Creación de ventanas en un entorno no grafico (MS-DOS)
- Programación orientada a los objetos (Delphi)

C – 1972

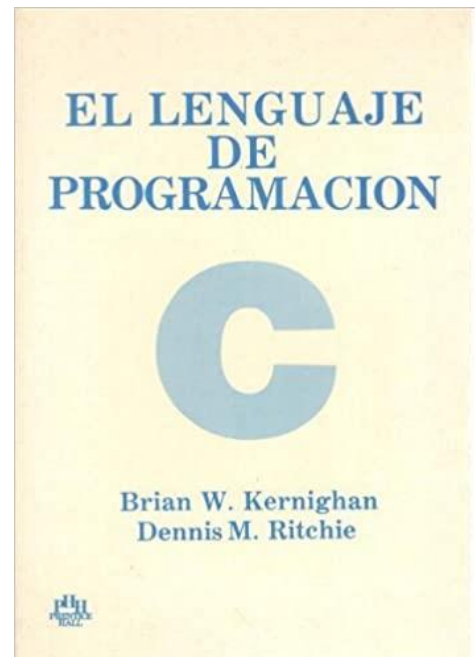
Los objetivos de este lenguaje de programación fueron orientados a la implementación de sistemas operativos específicamente de Unix. En esos momentos la búsqueda hacia un lenguaje capaz de correr el sistema operativo con ideas revolucionarias a la problemática de lentitud y coste en las computadoras. Reescrito con este nuevo lenguaje, más versátil debido a su fácil modificación con esto el código era más conciso y compacto.

Este lenguaje permite ser un intermediario entre diferentes lenguas de programación debido a su estructura, por esto mismo que se puede escribir en lenguaje de bajo nivel y por ende pueda desarrollar compiladores, está en una gran variedad de plataformas.

FUNCIONAMIENTO:

En concreto, la sintaxis de C cuenta con diferentes reglas y procesos que lideran la estructura de un programa. Cabe destacar que el lenguaje de programación C diferencia mayúsculas de minúsculas, por lo que se programa en letras minúsculas.

Además, la unidad mínima del lenguaje de programación C es el token. Los tokens son pequeños bloques de código que pueden ser variables, constantes, palabras clave, identificadores.



8 Autores: Brian Kernighan y Dennis Ritchie
(creador de C y co-creador de Unix).

Etapas	Procesos	Entrada	Salida	Descripción
1	Escritura del código	Idea del programador	Archivo fuente (.c)	Se escribe el código en un editor de texto como VS Code o Notepad++ y se guarda.
2	Preprocesamiento	Archivo fuente (.c)	Código expandido	El preprocesador lee el código, incluye las librerías necesarias y reemplaza las macros definidas.
3	Compilación	Código expandido	Archivo objeto (.obj / .o)	Un compilador (GCC o Clang) traduce el código a ensamblador y luego a lenguaje de máquina.
4	Enlazado	Archivo objeto (.obj / .o)	Archivo ejecutable (.exe)	El enlazador une tu archivo objeto con las

				librerías estándar del sistema para crear el programa final.
5	Ejecución	Archivo ejecutable	Resultado en pantalla / acción	El sistema operativo carga el ejecutable en la memoria RAM y la CPU procesa las instrucciones.

Tabla 1 Procesos del lenguaje C

Lenguas derivadas de C:

- C++
- Objective-C

Quinta Generación

1980

LENGUAJE DE ALTO NIVEL

"Un lenguaje de alto nivel es un lenguaje de programación diseñado para facilitar que las personas lo comprendan y escriban. Está más cerca de un lenguaje natural y utiliza comandos e instrucciones más abstractos, menos dependientes de los detalles específicos de la infraestructura de hardware de las computadoras." LEVONO

C++ – 1983

Es de propósito general, es una extensión del lenguaje C de 1975, creado por Bjarne Stroustrup, diseñada para ofrecer un mayor control de los recursos del sistema más la eficiencia de C. Creada para las nuevas capacidades como la orientación de objetos (*se basa en la idea de que los programas se pueden organizar como una colección de objetos interconectados, cada uno con su propio conjunto de datos y funcionalidades*) gracias a esto permite a los programadores organizar su código en clases y objetos, facilitando la creación de programas más modulares y reutilizables.

FUNCIONAMIENTO:

Las funciones son invocadas desde otras funciones, con una excepción: la función global *main()*, que tienen todos los programas en C++. Permite al compilador conocer dónde

```

PROCEDURE CreateCRC32Table( PolyHi, PolyLo : BITSET );
VAR
  I, J : CARDINAL;
  TempHi,
  TempLo : CARDINAL;
  DoXOR : BOOLEAN;
BEGIN
  FOR I := 0 TO 255 DO
    TempHi := 0;
    TempLo := I;
    FOR J := 1 TO 8 DO
      IF (TempLo MOD 2 = 1) THEN
        DoXOR := TRUE;
      ELSE
        DoXOR := FALSE;
      END;
      TempLo := TempLo >>1;
      IF (TempHi MOD 2 = 1) THEN
        TempLo := TempLo+08000H;
      END;
      TempHi := TempHi >>1;
      IF DoXOR THEN
        TempHi := CARDINAL(BITSET(TempHi)/PolyHi);
        TempLo := CARDINAL(BITSET(TempLo)/PolyLo);
      END;
    END;
    CRCtab[I].hi := BITSET(TempHi);
    CRCtab[I].lo := BITSET(TempLo);
  END;
END CreateCRC32Table;

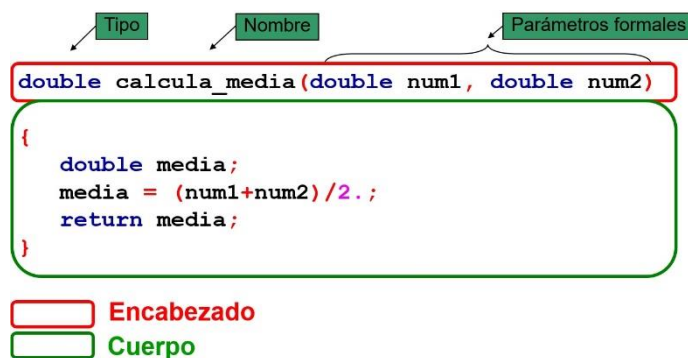
```

9 Método usado para detectar errores en la transmisión de datos o archivos (como en ZIP o Ethernet).

está el punto inicial de un programa. Por razones obvias, *main()* no puede ser invocada desde otras funciones.

DEFINICIÓN	DECLARACIÓN	LLAMADA
Es el código que realiza las tareas para las que la función ha sido prevista. La primera línea de la definición recibe el nombre de encabezamiento y el resto, un bloque encerrado entre llaves, es el cuerpo de la función.	una declaración permite al compilador: conocer de antemano las necesidades de memoria (se enviarán a la función dos double y se recibirá un double) gracias al tipado estático, detectar errores en tiempo de compilación	Se hace especificando su nombre y, entre paréntesis, las expresiones cuyos valores se van a enviar como argumentos de la función. Estos parámetros pueden ser identificadores o cualquier otra expresión válida.
EJEMPLO:		

Tabla 2 DEFINICIÓN, DECLARACIÓN Y LLAMADA LENGUAJE C++



```
double calcula_media(double x, double y);
```

10 Declaración

```
double resultado = calcula_media(numero1, numero2);
```

11 Llamada

12 Definición

¿Dónde se encuentra el lenguaje C++?

- Electrodomésticos inteligentes: En áreas específicas de control de ciclo y temporizadores como lavadoras, secadoras y hornos de microondas; en regulación de temperatura, alarmas y control de presión en neveras y refrigeradores y cafeteras; y en el mapeo de habitaciones como las aspiradoras.
- Sistemas de control industrial

1990

Python – 1991

Python se caracteriza por su legibilidad de su código, es mutiparadigmatica, ya que controla la prog, imperativa, la orientación de objetos como C++ hasta la prog. Funcional.

```
nombre : letter( letter | digit | "_" )*  
letra : "a" ... "z" | "A" ... "Z"  
dígito : "0" ... "9"
```

13Visualización de Python ordenes básicas

Es de código abierto y multiplataforma, creado por Guido Van Rossum. Con esta modalidad Python fue abierto al mundo, creando así múltiples actualizaciones en poco tiempo, de pasar de tener la herencia, manejo de excepciones, funciones y tipos modulares hasta llegar a la programación funcional lambda, reduce, filter y map. Buscaba que la programación fuera accesible para personas con niveles básicos de programación.

Cada regla comienza con un nombre (que identifica la regla que se está definiendo) seguido de dos puntos, `:`.

FUNCIONAMIENTO:

Es un lenguaje interpretado, quiere decir que ejecuta directamente el código línea a línea, que pasa con este funcionamiento que si hay un error en el código el programa directamente se detiene, con esto es fácilmente detectable el error. A diferencia de Pascal que se necesitaba determinar las variables Python los determina en el tiempo de ejecución haciendo el proceso de programación mucho más rápido. Además de eso deja de lado los símbolos complejos y los sustituye por los espacios en blanco siendo estos la indentación del código.

Python es útil para escribir código del lado del servidor debido a que ofrece muchas bibliotecas que constan de código preescrito para crear funciones de *backend* complejas.

Ventajas de Python:

- Sintaxis básica similar a la del inglés
- Se puede integrar fácilmente a otras lenguas de programación
- Accesible por sus múltiples tutoriales
- Es un lenguaje de scripting.

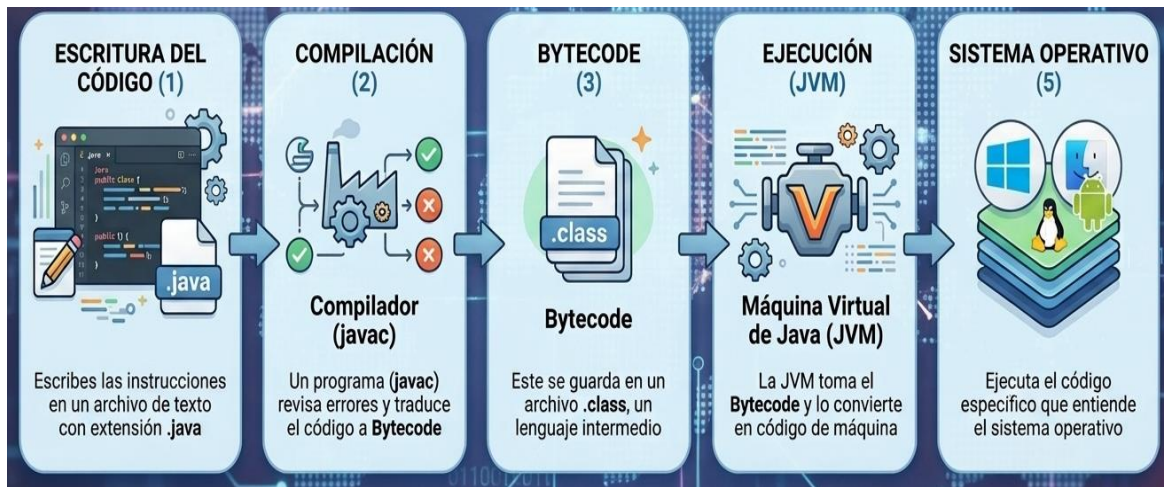
Para que lo usen los desarrolladores de Software:

- Realizar un seguimiento de los errores en el código del software
- Crear el software de forma automática
- Administrar los proyectos de software
- Desarrollar prototipos de software
- Desarrollar aplicaciones de escritorio por medio de las bibliotecas de interfaz gráfica de usuario (GUI)
- Desarrollar juegos simples basados en texto a videojuegos más complejos

Java – 1995

Java fue desarrollado por James Gosling en Sun Microsystem, derivado en su mayor parte de C y C++, sus aplicaciones son compiladas en bytecode (espera recibir para posteriormente ser compiladas a lenguaje de máquina mediante un compilador JIT a la hora de su ejecución) Este tuvo varias salidas o metas, finalmente redirigida a la web por la llegada de Mosaic (primer navegador web gráfico, funcionaba sobre el sistemas de Unix). Con Java publicado al público en el año 1996 con el objetivo de poner en marcha los decodificadores para una televisión interactiva.

FUNCIONAMIENTO:



14 Proceso de la programación de Java

Siguiendo esto entraria en juego el JVM, JRE Y JDK:

- JVM: Es el motor de ejecución del bytecode, es el que traduce las instrucciones al SO en tiempo real.
- JRE: Es el entorno, donde esta el JVM y la biblioteca.
- JDK: Es el que contiene el JRE para poder ejecutarlo, el compilador y las demas utilidades.

Ventajas de Java:

- Con Java, no es necesario escribir código aparte para Mac, Linux o Windows. En cambio, Java es un lenguaje basado en software. Su código se compila, se convierte en bytecode y se ejecuta en cualquier plataforma.
- Java se considera arquitectura neutral porque su intérprete de bytecode puede utilizarse en cualquier plataforma.

- La seguridad del lenguaje de programación Java es una de sus características más destacadas. Por defecto, Java ofrece varias capas de seguridad que permiten al desarrollador crear y ejecutar entornos de codificación libres de virus.

¿Dónde se encuentra java?

- Airbnb
- Uber
- Spotify
- Amazon
- eBay
- Netflix

spotify-web-api-java/

spotify-web-api-java

A Java wrapper for Spotify's Web API.



 53
Contributors

 2k
Used by

 54
Discussions

 1k
Stars

 290
Forks



15 Spotify usa Oracle Java SE

Años 2000 y Actualidad

2000

Scala – 2004

Scala, un lenguaje de programación flexible, fue creado por Martin Odersky con el objetivo de refinar y solucionar algunos problemas encontrados en Java. Es importante saber que, si bien Scala funciona con la máquina virtual Java (JVM), lo que le permite utilizar los recursos existentes de Java, es su propio lenguaje, no solo un complemento de Java. Scala se introdujo por primera vez como lengua independiente en 2004.



Lo que hace que Scala sea especial es su combinación de estilos de programación funcional y orientada a objetos, todo dentro de un sistema que verifica si hay errores durante la compilación. Esta combinación permite a los programadores escribir código que sea claro y funcione bien. Dado que Scala utiliza la JVM, tienes la ventaja de ejecutar tus programas Scala en muchos sistemas diferentes sin necesidad de volver a compilar el código cada vez.

FUNCIANAMIENTO:

Scala utiliza escritura estática, lo que significa que el compilador verifica los tipos de variables y expresiones en el momento de la compilación. Esto ayuda a detectar errores relacionados con el tipo en las primeras etapas del proceso de desarrollo, antes de ejecutar el programa. Encontrar y corregir errores antes ahorra tiempo y reduce la probabilidad de que surjan problemas inesperados

Ventajas:

1. Combina programación orientada a objetos y funcional
2. Se ejecuta en la JVM, compatible con Java
3. Potente coincidencia de patrones y sintaxis concisa
4. Ideal para concurrencia y sistemas distribuidos
5. Fuerte comunidad en big data y servicios backend

Scala es un lenguaje que muchos desarrolladores aprecian porque les permite crear aplicaciones que pueden expandirse fácilmente y que no cuestan una fortuna mantener en funcionamiento. Una cosa a tener en cuenta es que el núcleo de Scala podría no tener todas las funciones que necesitas de inmediato. Afortunadamente, existen marcos que simplifican el proceso de comenzar a programar en Scala.

Es una herramienta poderosa que combina lo mejor de la programación funcional y orientada a objetos. Con su sintaxis concisa, escritura sólida e integración Java perfecta, Scala es ideal para crear aplicaciones escalables y de alto rendimiento. No importa si trabajas en big data, desarrollo backend o sistemas reactivos, aprender Scala puede abrir un mundo de posibilidades de programación modernas y eficientes.

¿Dónde se Utiliza?

El lenguaje Scala se utiliza en una variedad de redes sociales, Plataformas de Streaming:

- Netflix
- Disney
- LinkedIn
- Airbnb
- Twitter/X

Clojure – 2007

Clojure fue creado por Rich Hickey en 2007 con la intención de ofrecer un lenguaje funcional moderno que pudiera integrarse con el código legado de Java. Su enfoque se basa en la filosofía de minimizar el estado mutable y facilitar el trabajo concurrente.

A diferencia de muchos lenguajes funcionales anteriores, Clojure fue diseñado desde el principio para integrarse sin fricciones con la máquina virtual de Java (JVM), lo que le permitió



```
1 (ns main)
2
3 (def my-name "Some other name")
4
5 (defn hello-message
6   [name]
7   (str "Hello " name "!"))
8
9 (defn -main
10  [& args]
11  (println (hello-message my-name)))
12
13 (defn article-url
14  [article-id]
15  (str "https://awesome-website.com/articles/"))
```

16 Define una variable global, crea una función de saludo y configura el punto de inicio del programa

aprovechar el ecosistema existente de bibliotecas y herramientas Java, además de facilitar su adopción en entornos empresariales ya establecidos.

Clojure nació de la necesidad de contar con un lenguaje funcional moderno, práctico y poderoso, con énfasis en la simplicidad, inmutabilidad y concurrencia, todo ello sin renunciar a la compatibilidad con infraestructuras existentes.

Ventajas:

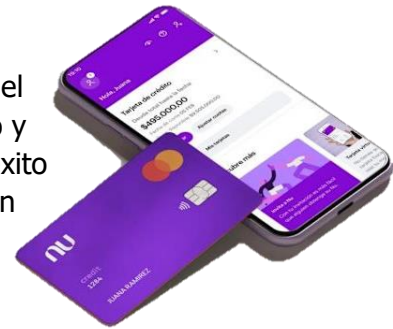
- Sintaxis concisa y expresiva: Clojure utiliza una sintaxis minimalista basada en listas y expresiones S-expr que reducen la cantidad de código necesario para expresar ideas complejas. Esto no solo agiliza el desarrollo, sino que también mejora la legibilidad y mantenibilidad del código a largo plazo.
- Paradigma funcional por defecto: A diferencia de muchos lenguajes que incorporan funcionalidad, pero no la priorizan, en Clojure todo gira alrededor de funciones puras, inmutabilidad y ausencia de efectos secundarios. Esto facilita razonar sobre el código, evita errores comunes relacionados con el estado mutable y fomenta un estilo declarativo y predecible.
- Modelo de concurrencia avanzado y seguro: Clojure proporciona abstracciones como refs, atoms, agents y core.async para manejar concurrencia y estado compartido de forma segura y sin bloquear el sistema. Esto permite construir aplicaciones concurrentes robustas sin la complejidad habitual asociada a los hilos y sincronización manual.
- Interoperabilidad total con Java: Ejecutándose sobre la JVM, Clojure puede acceder de manera directa y sencilla a cualquier librería Java existente, desde utilidades estándar hasta frameworks empresariales. Esto posibilita la modernización de sistemas legacy y el aprovechamiento del vasto ecosistema Java sin sacrificar las ventajas de la programación funcional.
- REPL interactivo y poderoso: El REPL (Read-Eval-Print Loop) de Clojure es una herramienta fundamental que permite a los desarrolladores probar fragmentos de código, explorar APIs y depurar de forma dinámica. Esta interactividad acelera el ciclo de desarrollo y mejora la productividad al ofrecer feedback inmediato y permitir un enfoque exploratorio.

Clojure es un entorno dinámico con el que puedes interactuar. Casi todas las construcciones del lenguaje están cosificadas y, por lo tanto, pueden examinarse y modificarse. Puede hacer crecer su programa, con datos cargados, agregando funciones, corrigiendo errores, probando, en un flujo ininterrumpido. También proporciona las herramientas para evitar estados mutables, proporciona funciones como objetos de primera clase y enfatiza la iteración recursiva en lugar de bucles basados en efectos secundarios. Clojure es impuro, pero respalda la filosofía de que los programas que son más funcionales son más sólidos.

Simplifica la programación multiproceso de varias maneras. Debido a que las estructuras de datos centrales son inmutables, pueden compartirse fácilmente entre subprocesos. Clojure, al ser un lenguaje práctico, permite cambiar el estado, pero proporciona un mecanismo para garantizar que, cuando lo hace, se mantenga consistente, al tiempo que alivia a los desarrolladores de tener que evitar conflictos manualmente mediante bloqueos, etc.

¿Dónde se aplica?

Nubank es una de las plataformas de banca digital más grandes del mundo y atiende a más de 100 millones de clientes en Brasil, México y Colombia. Desde sus inicios, Nu ha tenido una historia de éxito espectacular con Clojure, ejecutando software de misión crítica a gran escala. En 2020, la empresa adquirió Cognitect, la consultora detrás de Clojure y la base de datos Datomic.



Dart – 2011

Dart es un lenguaje de programación open source, relativamente nuevo, que fue desarrollado por Google y que lanzó su primera versión en 2011. Este lenguaje se creó con el objetivo de permitir a los desarrolladores utilizar un lenguaje orientado a objetos y con análisis estático de tipo.

La programación Dart es una alternativa a reemplazar JavaScript y convertirse en el lenguaje prioritario para los navegadores actuales, aunque este lenguaje todavía está en proceso de mejoras y adaptaciones.

FUNCIONAMIENTO:

La principal diferencia que presenta Google Dart frente a otros lenguajes de programación es que es más fácil de leer ya que la sintaxis de este se acerca al lenguaje humano, tiene más posibilidades y menos comandos.

Es posible utilizar caracteres en blanco, pestañas y saltos de línea a discreción, lo que permite a los programadores de estructurar bien el código, de manera que sea últimamente ignorado por el compilador. Sin embargo, hay unas cuantas excepciones, como las palabras clave y los nombres de variables y funciones, es decir, todas las definiciones establecidas dentro del lenguaje Dart. Entre ellas se encuentran, por ejemplo, "if", "else", "string" o "void".

Por último, se diferencia entre mayúsculas y minúsculas, lo que deja espacio a las denominaciones.

Ventajas:

- Es de acceso gratuito para cualquier persona.
- Detrás de su programación se encuentra Google, lo que ofrece perspectivas a largo plazo para el desarrollo del lenguaje.

- Dart es fácil de aprender debido a que los desarrolladores han simplificado características complicadas de otros lenguajes.
- Funciona en todos los navegadores móviles y de escritorio actuales.

Características:

- Programación estructurada y flexible: Google diseñó Dart para poder ser utilizado en proyectos de una sola persona hasta proyectos más desarrollados o complejos.
- Lenguaje familiar y fácil de aprender: Es un lenguaje realmente sencillo y fácil de aprender. En su sitio web se pueden encontrar varios tutoriales, y también permite colaboraciones de otros desarrolladores.
- Permite la adaptación de nueva herramienta a cualquier navegador web: El lenguaje de programación Dart se puede ejecutar de dos maneras; en una máquina virtual (MV), o en un motor de Javascript utilizando un compilador para traducir el código. Esto le permite adaptarse a cualquier navegador.
- Lenguaje basado en clases e interfaces o POO: Gracias a sus basamentos en clases o en la programación orientada a objetos, se facilita la encapsulación y la reutilización del código.

¿Dónde se utiliza?

Varias aplicaciones de Google se reescribieron para adaptarse al lenguaje de DART

- Google Pay: Su plataforma global de pagos digitales se reescribió completamente en Dart para unificar las versiones de Android e iOS.
- Google Earth: La herramienta de exploración planetaria aprovecha el motor gráfico de Dart y Flutter para renderizar mapas fluidos.
- Google Ads: La aplicación móvil que usan millones de empresas para gestionar sus campañas publicitarias.
- NotebookLM: La innovadora herramienta de notas e investigación asistida por Inteligencia Artificial de Google.
- YouTube Create: Su aplicación de edición de video móvil para creadores de contenido.



Análisis crítico

1. ¿Cuál fue el avance más importante dentro de la temática seleccionada?

El avance más crítico fue la transición de la configuración física del hardware a la abstracción del software y la portabilidad universal. En los inicios con la ENIAC, permitieron que el código dejara de depender de una máquina específica para volverse un traductor universal.

2. ¿Cómo transformó ese avance la forma de crear software?

Este avance transformó el desarrollo de varias maneras fundamentales:

- Tareas de cálculo que tomaban 12 horas manualmente pasaron a realizarse en 30 segundos con la ENIAC.
- Los programadores dejaron de conectar cables para escribir notación simbólica y expresiones matemáticas lo que permitió crear software modular, reutilizable y más fácil de mantener.
- La evolución del hardware permitió el concepto de tiempo compartido, donde la CPU puede ejecutar múltiples tareas simultáneamente mediante un planificador.

3. ¿Qué herramientas o tecnologías actuales existen gracias a esa evolución?

La evolución de estos lenguajes se sustenta gran parte de la tecnología moderna:

- Unix: Sistema operativo multitarea y multiusuario
- Electrodomésticos e Industria
- Para manejar sus sistemas distribuidos para sus servicios financieros de misión crítica.
- La unificación multiplataforma.

4. ¿Qué competencias debe desarrollar hoy un programador para adaptarse a estos cambios?

- Programación funcional (clave característica en lenguajes como Scala y Clojure).
- Capacidad para escribir código que maneje múltiples procesos de forma segura y sin errores de estado mutable.
- Habilidad para usar herramientas como Dart/Flutter que permiten desplegar software en Android, iOS y la web simultáneamente.

5. ¿Qué tendencias futuras podrían transformar el desarrollo de software durante los próximos 10 años?

- Unificación Total de Plataformas: Idiomas como Dart buscan reemplazar a JavaScript y convertirse en el estándar prioritario para navegadores y aplicaciones móviles, eliminando la necesidad de escribir código diferente para cada sistema.

CONCLUSIÓN

Los lenguajes de programación desde 1940 hasta la actualidad es un proceso de abstracción creciente, democratización del acceso y búsqueda de la eficiencia multiplataforma.

Los puntos clave que resumen esta evolución son:

- La programación comenzó con la configuración manual de cables y tuberías de vacío en la ENIAC , donde una tarea tomaba horas de ajuste físico. Como Python y Dart, cuya sintaxis es cercana al lenguaje humano, permitiendo que el programador se concentre en la lógica y no en los detalles del hardware.
- Uno de los mayores retos fue superar la dependencia del hardware. Inicialmente, el código escrito para una máquina no funcionaba en otra. Soluciones como la Máquina-P de Pascal y, más tarde, la Máquina Virtual de Java (JVM) , permitieron el concepto de "escribir una vez, ejecutar en cualquier lugar", lo que facilitó la expansión del software a nivel global.
- (C++, Java) permitió crear software modular y reutilizable para manejar de forma eficiente la concurrencia y los grandes volúmenes de datos.
- nació con el objetivo explícito de abrir la informática a estudiantes y no solo a científicos. Esta tendencia continúa hoy con Python y Dart.
- Impacto en la vida cotidiana: Lo que comenzó como una herramienta para calcular tablas de artillería, hoy sustenta toda la infraestructura moderna.

En resumen, la historia de los lenguajes de programación es el reflejo de cómo la humanidad ha logrado dominar la complejidad técnica para convertir a las computadoras en herramientas universales, accesibles y esenciales para cada aspecto de la sociedad contemporánea.

INFOGRAFÍA

- BASIC historia y desarrollo
https://es.wikipedia.org/wiki/Historia_de_los_lenguajes_de_programaci%C3%B3n
- Circuito integrado
<https://www.ibm.com/es-es/think/topics/central-processing-unit-history>
https://es.wikipedia.org/wiki/Circuito_integrado
- Historia del Hardware
[https://es.wikipedia.org/wiki/Historia_del_hardware_de_computadora_\(1960-presente\)](https://es.wikipedia.org/wiki/Historia_del_hardware_de_computadora_(1960-presente))
- Sheduler
https://academicos.azc.uam.mx/jestrada/archivos/so/tema4/expo_planificador.pdf
- Programación estructurada
https://es.wikipedia.org/wiki/Programaci%C3%B3n_estructurada
- Lenguaje tipeado
https://es.wikipedia.org/wiki/Tipado_fuerte
- Lenguaje pascal
<https://museo.inf.upv.es/pascal/>
- Lenguaje C
<https://www.nokia.com/bell-labs/about/dennis-m-ritchie/chist.html>
<https://immune.institute/blog/lenguaje-de-programacion-c/>
- Lenguaje c++
<https://openwebinars.net/blog/que-es-cpp/>
https://www2.eii.uva.es/fund_inf/cpp/temas/8_funciones/funciones.html
- Programación orientada a objetos
<https://profile.es/blog/que-es-la-programacion-orientada-a-objetos/>
- Python
<https://es.wikipedia.org/wiki/Python>
<https://docs.python.org/es/3/reference/introduction.html>
<https://aws.amazon.com/es/what-is/python/>
- Lenguaje de alto nivel
<https://www.lenovo.com/mx/es/glosario/lenguaje-de-alto-nivel/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsId=AfmBOopiUIWYVF3vVvkm3wdh40HMUoH9aNqo8lz0-F6tCBK1hqEi6t8Z>
- bytecode
https://es.wikipedia.org/wiki/Bytecode_Java
- Mosaic
<https://es.wikipedia.org/wiki/Mosaic>
- Java
https://www.coursera.org/in/articles/what-is-java-used-for?utm_medium=sem&utm_source=gg&utm_campaign=b2c_latam_x_multi_ftcof_career-academy_cx_dr_bau_gg_pmax_gc_s1-v2_en_m_hyb_24-04_x&campaignid=21239613742&adgroupid=&device=c&keyword=&matchtype=&network=x&devicemodel=&creativeid=&assetgroupid=6501905150&targetid=&extensionid=&placement=&gad_source=1&gad_campaignid=21320925518&gbraid=0AAAAADdKX6auEhHbmhFp_J-

[vwhxjhsj3T&qclid=Cj0KCQjw0JnRBhDJARIsALobnXaFM74FRJQe0NYDOA4byWvDGZxtUskN_gTo5IudehRYIObIbBG48LMArJpEALw_wcB](https://www.coursera.org/in/articles/what-is-java-used-for?utm_medium=sem&utm_source=gg&utm_campaign=b2c_latam_x_multi_ftcof_career-academy_cx_dr_bau_gg_pmax_gc_s1-v2_en_m_hyb_24-04_x&campaignid=21239613742&adgroupid=&device=c&keyword=&matchtype=&network=x&devicemodel=&creativeid=&assetgroupid=6501905150&targetid=&extensionid=&placement=&gad_source=1&gad_campaignid=21320925518&gbraid=0AAAAADdKX6auEhHbmhFp_J-)
https://www.coursera.org/in/articles/what-is-java-used-for?utm_medium=sem&utm_source=gg&utm_campaign=b2c_latam_x_multi_ftcof_career-academy_cx_dr_bau_gg_pmax_gc_s1-v2_en_m_hyb_24-04_x&campaignid=21239613742&adgroupid=&device=c&keyword=&matchtype=&network=x&devicemodel=&creativeid=&assetgroupid=6501905150&targetid=&extensionid=&placement=&gad_source=1&gad_campaignid=21320925518&gbraid=0AAAAADdKX6auEhHbmhFp_J-
[vwhxjhsj3T&qclid=Cj0KCQjw0JnRBhDJARIsALobnXaFM74FRJQe0NYDOA4byWvDGZxtUskN_gTo5IudehRYIObIbBG48LMArJpEALw_wcB](https://www.coursera.org/in/articles/what-is-java-used-for?utm_medium=sem&utm_source=gg&utm_campaign=b2c_latam_x_multi_ftcof_career-academy_cx_dr_bau_gg_pmax_gc_s1-v2_en_m_hyb_24-04_x&campaignid=21239613742&adgroupid=&device=c&keyword=&matchtype=&network=x&devicemodel=&creativeid=&assetgroupid=6501905150&targetid=&extensionid=&placement=&gad_source=1&gad_campaignid=21320925518&gbraid=0AAAAADdKX6auEhHbmhFp_J-)

- ENIAC
<https://es.wikipedia.org/wiki/ENIAC>
- Plankalkul:
<https://administraciondesistemas.com/plankalkul-primer-lenguaje-programacion-historia/>
<https://www.computer.org/csdl/magazine/an/1997/02/man1997020017/13rRUIJcWqp>
 BINIAC
<https://maquiasmecanicas.blogspot.com/2014/10/binac-binary-automatic-computer.html>
- COBOL
<https://www.ibm.com/es-es/think/topics/cobol>
- Bloques de códigos
<https://www.ibm.com/docs/es/spss-modeler/saas?topic=scripting-blocks-code>
- Compilador
<https://www.ibm.com/mx-es/think/topics/compiler>
- ALGOL
<https://es.wikipedia.org/wiki/ALGOL>
https://en.wikipedia.org/wiki/ALGOL_60
<https://www.algol60.org/1home.htm>
<https://cs.baylor.edu/~maurer/SieveE/Algol60.htm>
- Clojure
<https://clojure.org/>
- Scala
<https://www.igmguru.com/blog/what-is-scala>
- DART
<https://dart.dev/overview>
<https://www.hiberus.com/crecemos-contigo/que-es-el-lenguaje-de-programacion-dart/>